Apuntes · Clase 36 · Boosting

Para leer antes de los ejercicios · lectura de ~45 min

Lo más importante, en 6 ideas

1. **Secuencia.** Los árboles se entrenan uno tras otro; cada uno recibe información del error del anterior.
2. **Aditivo.** El modelo final es una suma ponderada: $F = F_0 + \eta \cdot h_1 + \eta \cdot h_2 + \dots$
3. **AdaBoost.** Sube el peso de los casos mal clasificados y vota con peso $\alpha = \tfrac{1}{2} \ln((1 - \varepsilon)/\varepsilon)$.
4. **Gradient Boosting.** Cada árbol predice el gradiente negativo de la pérdida (el residuo, si es cuadrática).
5. **η y n_estimators.** Van juntos: menor η, más árboles. Se elige n con early stopping.
6. **XGBoost y LightGBM.** Gradient Boosting regularizado y rápido; LightGBM crece por hoja y con histogramas.

Cada sección trae una explicación, un **ejemplo resuelto con números distintos** a los de los ejercicios y una **pista** para el notebook `C36_Boosting.ipynb` o el cuaderno `C36_Ejercicios_Cuaderno_Boosting.pdf`.

1 Bagging y boosting: dos ideas distintas

En el bosque (Clase 35) cada árbol se entrena con su propia muestra, sin importarle los demás: por eso se pueden entrenar en paralelo, y promediarlos baja la **varianza**. En boosting el árbol 2 necesita saber dónde falló el árbol 1, así que se entrena **después**, sobre los **mismos datos completos**, y sus contribuciones se **suman**. El objetivo típico es bajar el sesgo y el error.

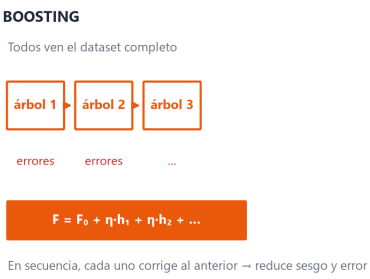
Ejemplo resuelto · el costo de ser secuencial

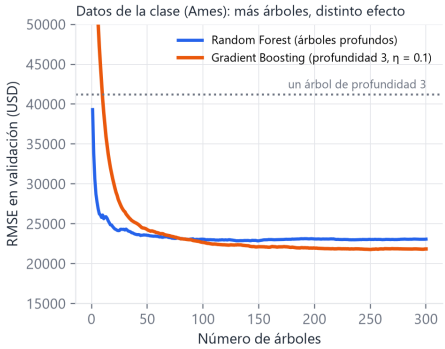
Un modelo de 600 árboles tarda 1 s por árbol y dispones de 12 procesadores.

Familia	Tiempo aproximado	Razón
Bosque	600 / 12 = 50 s	los árboles son independientes: se reparten
Boosting	600 s	cada árbol espera al anterior

Por eso XGBoost invierte tanto en paralelizar *dentro* de cada árbol (la búsqueda de cortes), aunque entre árboles siga siendo secuencial.







Pista para la Parte A del notebook y el Bloque 1 del cuaderno

Compara las curvas de error al agregar árboles: el bosque se estanca pronto; boosting sigue bajando. Pregúntate qué ve cada árbol y qué hace con ello.

2 El modelo aditivo y el ciclo de boosting

Todos los modelos de la clase siguen cinco pasos: **inicializar** con una predicción base F_0, **medir el error**, **ajustar** un nuevo aprendiz, **actualizar** el modelo sumando su contribución y **detener** tras M etapas o cuando la validación deja de mejorar. Lo que separa a AdaBoost de Gradient Boosting es solo la definición de «error».

Ejemplo resuelto · predecir una casa con tres árboles

$F_0 = 160$ (miles de USD, el promedio). Para una casa, los tres árboles predicen $h_1 = +50$, $h_2 = +20$, $h_3 = -10$. $\eta = 0.3$.

Etapa	Cálculo	Predicción F
0	—	160
1	160 + 0.3 · 50	175
2	175 + 0.3 · 20	181
3	181 + 0.3 · (−10)	178

Cada árbol aporta solo una fracción η de lo que predice: es la «tasa de aprendizaje».

3 AdaBoost: pesos, error y voto

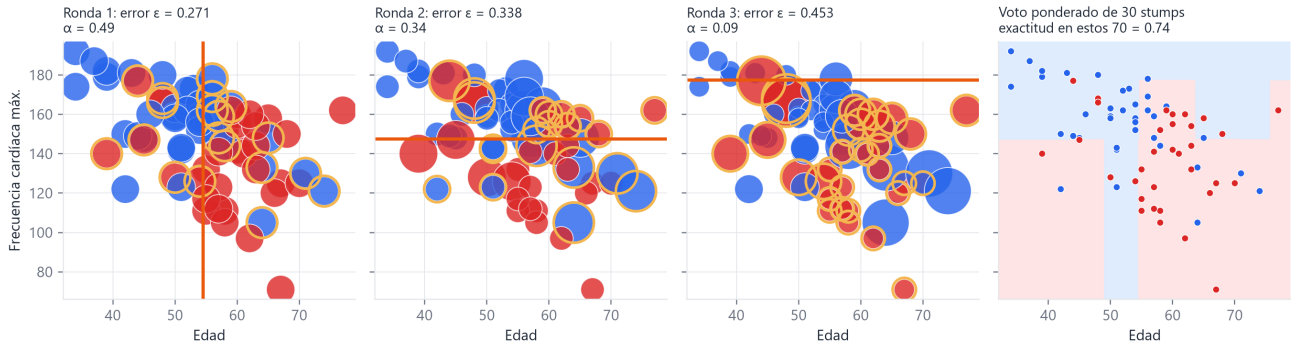
AdaBoost trabaja con clases −1/+1. Empieza con pesos iguales 1/n. Cada ronda: entrena un árbol (normalmente un *stump*) con esos pesos, calcula su error **ponderado** ε, su peso $\alpha = \tfrac{1}{2}\ln((1-\varepsilon)/\varepsilon)$ y actualiza los pesos: los errores se multiplican por e^{α} y los aciertos por $e^{-\alpha}$. Después se normaliza.

Ejemplo resuelto · una ronda con 10 observaciones

Pesos iniciales 0.1. El stump falla en 2 observaciones: $\varepsilon = 0.2$.

Paso	Cálculo	Resultado
α	$\tfrac{1}{2}\ln(0.8/0.2) = \tfrac{1}{2}\ln 4$	0.693
Factor de los errores	$e^{0.693}$	2
Factor de los aciertos	$e^{-0.693}$	0.5
Pesos sin normalizar	errores 0.1·2 = 0.2 · aciertos 0.1·0.5 = 0.05	suma = 2·0.2 + 8·0.05 = 0.8
Pesos normalizados	errores 0.2/0.8 · aciertos 0.05/0.8	0.25 y 0.0625

Fíjate: tras normalizar, los 2 errores concentran el 50% del peso total (2 · 0.25). Siempre ocurre: el siguiente árbol ve los errores del anterior como «la mitad del problema». El voto final: tres árboles con α = 0.9, 0.4, 0.3 votan −1, +1, +1; la suma es −0.9 + 0.7 = −0.2, así que gana la clase −1 aunque haya más votos +1.



Tamaño del punto = peso al comenzar la ronda · borde amarillo = error del stump · rojo = enfermedad (AHD = Yes), azul = No

Pista para la Parte B del notebook y los Bloques 2 y 3 del cuaderno

Ordena el cálculo siempre igual: (1) ε sumando pesos de los errores, (2) α, (3) factor de errores e^{α} y de aciertos $e^{-\alpha}$, (4) normalizar. Si $\varepsilon > 0.5$, α sale negativo: el árbol se «invierte».

4 Gradient Boosting: ajustar residuos

Se empieza con F_0 = promedio de y (en pérdida cuadrática). En cada etapa se calculan los **residuos** $r = y - F$, se ajusta un árbol pequeño para predecirlos y se actualiza $F = F + \eta \cdot h$.

Ejemplo resuelto · una etapa con 4 observaciones

y = [20, 14, 18, 24] para A, B, C, D. F_0 = promedio = 19. El árbol separa {A, B} de {C, D}.

Obs.	y	F_0	$r = y - F_0$	h (media de r por hoja)	F_1 con η = 0.5	$y - F_1$
A	20	19	+1	−2 (media de +1, −5)	19 + 0.5·(−2) = 18	+2
B	14	19	−5	−2	18	−4
C	18	19	−1	+2 (media de −1, +5)	19 + 0.5·2 = 20	−2
D	24	19	+5	+2	20	+4

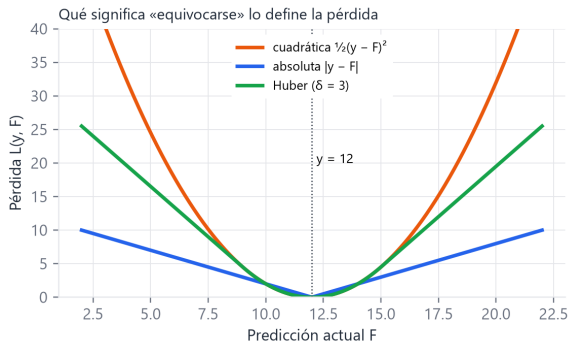
RMSE antes: $\sqrt{(1 + 25 + 1 + 25)/4} = \sqrt{13} = 3.61$. Después con η = 0.5: $\sqrt{(4 + 16 + 4 + 16)/4} = \sqrt{10} = 3.16$. Con η = 1 los nuevos residuos serían [+3, −3, −3, +3], RMSE 3.0: baja más rápido, pero «de golpe». El siguiente árbol estudiará los residuos [+2, −4, −2, +4].

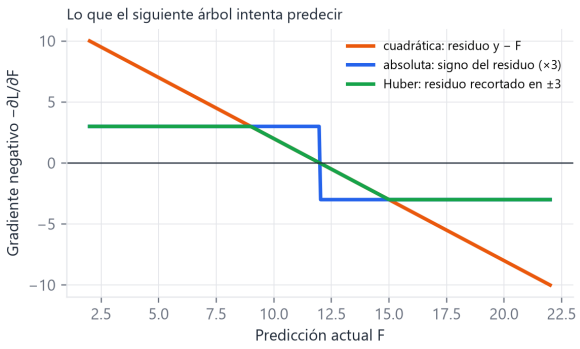
¿Qué es el «gradiente»? La pérdida L(y, F) mide qué tan mal predice F. El gradiente negativo $-\partial L/\partial F$ indica hacia dónde mover F para reducir la pérdida; el nuevo árbol aprende a predecir esa dirección. Para $L = \frac{1}{2}(y - F)^2$ es $y - F$. Para la pérdida absoluta es el signo del residuo, y para Huber, el residuo recortado: robustos a valores extremos.

Ejemplo resuelto · qué predice el siguiente árbol según la pérdida

Pérdida	y, F	$-\partial L/\partial F$
Cuadrática $\frac{1}{2}(y - F)^2$	y = 12, F = 9	$y - F = +3$
Absoluta $\lvert y - F \rvert$	y = 12, F = 9	$\text{signo}(y - F) = +1$
Huber (δ = 2)	y = 12, F = 9	residuo recortado en ±2 → +2
Logística (y = 0 ó 1, $p = 1/(1+e^{-F})$)	y = 1, F = 0 → p = 0.5	$y - p = +0.5$
Logística	y = 1, F = 2 → p = 0.881	$y - p = +0.119$

Con la logística, a medida que el modelo se acerca a la clase correcta el gradiente se achica solo.





Pista para la Parte C del notebook y los Bloques 4 a 6 del cuaderno

En el notebook programas Gradient Boosting **desde cero** con DecisionTreeRegressor y compruebas que coincide con sklearn. Antes de empezar un ejercicio, escribe F_0, los residuos y la regla de actualización.

5 Tasa de aprendizaje y número de árboles

η es el tamaño del paso. El profesor lo explica con una analogía: estudiar poco cada día (η bajo, muchos «días» = árboles) deja más aprendizaje que estudiar todo la noche anterior. Con η bajo hacen falta más árboles, el costo sube, pero el modelo suele ser más estable. Un modelo idealizado ayuda a ver el efecto: si cada árbol corrigiera exactamente la fracción η del residuo, tras m árboles quedaría $r_0(1 - \eta)^m$.

Ejemplo resuelto · ¿cuántos árboles hacen falta?

$r_0 = 10$. Queremos bajar el residuo a menos de 1 (a menos del 10% del inicial).

η	Residuo tras 1, 2, 3 árboles	Árboles para bajar de 1
0.5	$5 \cdot 2.5 \cdot 1.25$	$0.5^m < 0.1 \rightarrow m \geq 4$ (0.0625)
0.1	$9 \cdot 8.1 \cdot 7.29$	$0.9^m < 0.1 \rightarrow m \geq 22$ (0.098)

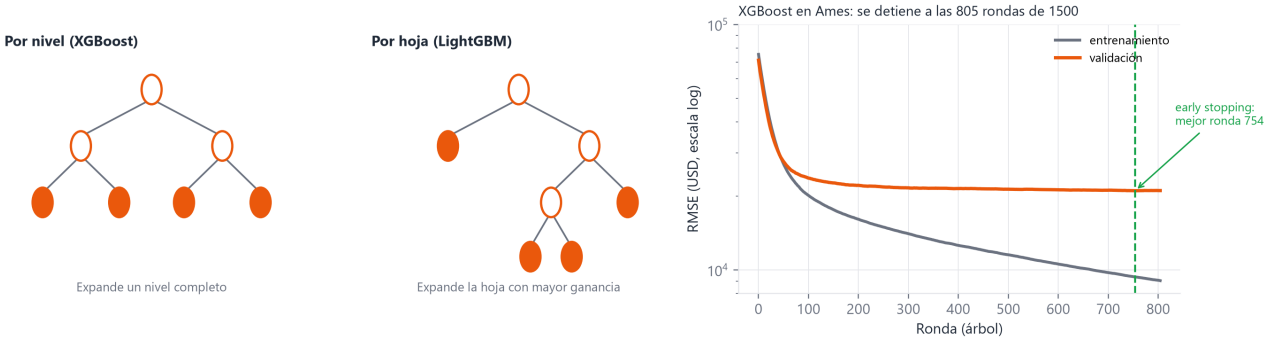
Dividir η entre 5 multiplica por ≈ 5.5 los árboles necesarios. En datos reales el óptimo se busca con validación: en Ames, η = 0.3 necesitó ≈ 460 árboles y η = 0.1, ≈ 590, con casi el mismo error mínimo (22.5k y 21.5k).

6 XGBoost y LightGBM

XGBoost (*Extreme Gradient Boosting*) es Gradient Boosting con ingeniería: una función objetivo con **regularización** (penaliza árboles complejos), gradientes de primer y segundo orden, submuestreo de filas y columnas como el bosque, tratamiento de valores nulos y de dispersión, y un núcleo en C++ (no está en scikit-learn: se instala aparte, aunque ofrece una interfaz compatible). **LightGBM** (*Light Gradient Boosting Machine*) usa **histogramas** (agrupa valores continuos en bins) y crece el árbol **hoja por hoja**: siempre expande la hoja con mayor ganancia, así que el árbol queda asimétrico y cuesta menos con muchos datos. Con datos pequeños el crecimiento hoja por hoja puede sobreajustar.

Ejemplo resuelto · hojas y profundidad

¿Cuántas hojas tiene como máximo un árbol por nivel con max_depth = 4? $2^4 = 16$. En LightGBM con num_leaves = 15 el árbol puede ser asimétrico: 15 hojas implican 14 cortes, y en el caso extremo una rama de profundidad 14 (por eso conviene fijar también max_depth o min_data_in_leaf).
Regla práctica: num_leaves < $2^{\text{max_depth}}$. Con max_depth = 5, num_leaves debe ser menor que 32.



Early stopping. Se reserva un conjunto de validación (distinto del test), se mide el error después de cada árbol y se detiene cuando no mejora durante k rondas consecutivas. El número de árboles se elige solo, y se conserva la mejor ronda.

Ejemplo resuelto · early stopping con paciencia 2

RMSE de validación por ronda: 30, 25, 22, 21, 21.4, 21.9, 22.5.
El mínimo es 21 en la ronda 4. Las rondas 5 y 6 no mejoran (2 seguidas): con paciencia 2 se detiene tras la 6 y la mejor ronda es la 4.

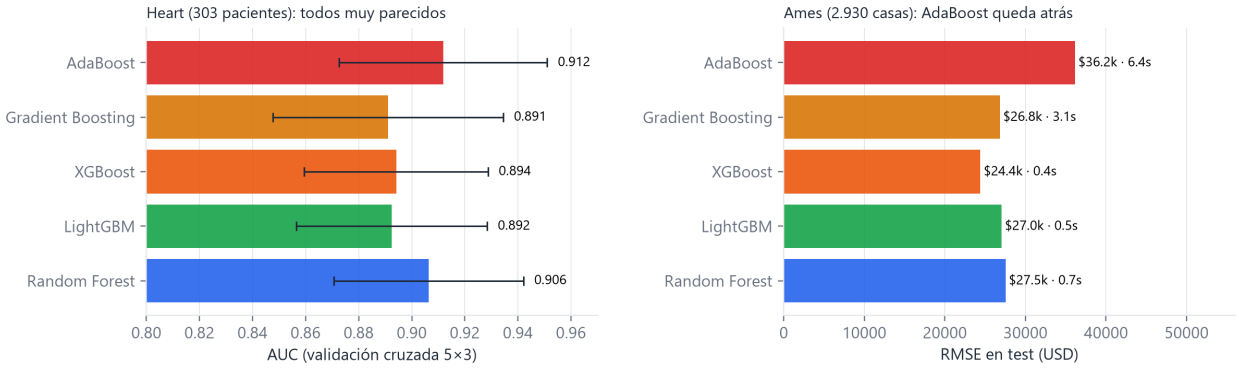
Pista para la Parte D del notebook y los Bloques 7 y 8 del cuaderno

Reserva siempre un conjunto de validación propio para early stopping: si usas el test, tu estimación final deja de ser honesta.

7 Cómo elegir entre los cinco modelos

Si lo que domina tu problema es…	Elige	Porque
Una línea base simple, datos limpios	AdaBoost	intuición clara y pocos hiperparámetros
Control didáctico del proceso	Gradient Boosting	puedes seguir F_0, r_1, F_1… paso a paso
Rendimiento general (candidato por defecto)	XGBoost	regularizado y muy optimizado
Gran escala de filas y variables	LightGBM	histogramas y crecimiento por hoja
Muchísimas variables categóricas	CatBoost	las maneja de forma nativa (no se vio en la práctica)

Con los datos de la clase: en Heart (303 filas) los cinco modelos quedan entre 0.89 y 0.91 de AUC, dentro del desvío; en Ames (2.930 casas) XGBoost fue el mejor (RMSE 24.4k), seguido de Gradient Boosting (26.8k), LightGBM (27.0k) y el bosque (27.5k), y AdaBoost el peor (36.2k). En clase el profesor lo resumió así: «a problemas complejos, modelos complejos; a problemas simples, modelos simples», y «el consejo es probar».



8 Chuleta de código

Quiero…	Código
AdaBoost	AdaBoostClassifier(n_estimators=250, learning_rate=0.05, random_state=42)
Gradient Boosting	GradientBoostingRegressor(n_estimators=350, learning_rate=0.05, max_depth=2)
Ver F_1, F_2, … paso a paso	for pred in gb.staged_predict(X_val): …
XGBoost con early stopping	XGBRegressor(n_estimators=1500, learning_rate=0.05, early_stopping_rounds=50).fit(X, y, eval_set=[(Xv, yv)])
LightGBM	LGBMClassifier(num_leaves=15, subsample=0.8, subsample_freq=1, verbosity=-1)
Importancia correcta	permutation_importance(modelo, X_test, y_test, scoring='roc_auc', n_repeats=20)
Optimizar	RandomizedSearchCV(pipe, {'model__max_depth': […]}, n_iter=20, scoring='roc_auc', cv=cv)

9 Errores comunes

Error	Por qué está mal	Qué hacer
Usar boosting con etiquetas dudosas	Persigue los errores: se obsesiona con el ruido	Limpia y revisa etiquetas; prueba con η bajo
η bajo con pocos árboles	Subajusta	Más árboles o early stopping
Early stopping contra el test	El test deja de ser honesto	Validación aparte
Fiarse de feature_importances_ entre librerías	XGBoost = ganancia, LightGBM = cortes	Permutación sobre el test
LightGBM subsample sin subsample_freq	No hace nada	subsample_freq = 1
Declarar ganador con una partición	Diferencias dentro del desvío	CV repetida y media ± desvío

Lo que debes recordar

- Orden sugerido: estos apuntes → Bloques 1–3 del cuaderno → Partes A y B del notebook → Bloques 4–6 → Parte C → Bloques 7–9 → Parte D → ejercicio de cierre.
- Si te trabas más de 10 minutos, mira la solución, entiéndela y vuelve a hacerlo sin mirar.